AIML Summer Internship 2026

IIHMF, MNNIT Allahabad, Prayagraj

CAPSTONE PROJECT REPORT

Employee Attrition Prediction System

Submitted by

Team Member	Role
Archit Srivastava	Team Lead
Param Gupta	Team Member
Adish Tripathi	Team Member

B.Tech Computer Science & Engineering

United College of Engineering & Research (UCER), Prayagraj

Project 4 — Employee Attrition Prediction System

Table of Contents

Chapter 1: Introduction 3

1.1 Background 3
1.2 Problem Statement 3
1.3 Objectives 3
1.4 Business Impact of Attrition 3
1.5 Scope of this Project 4

Chapter 2: Literature Review 5

2.1 Existing Approaches 5
2.2 Research Studies 5
2.3 Research Gap Addressed 5

Chapter 3: Methodology 6

3.1 Dataset Description 6
3.2 Data Preprocessing 6
3.3 Algorithms Used 7
3.4 Handling Class Imbalance 7

Chapter 4: Implementation 8

4.1 Exploratory Data Analysis 8
4.2 Feature Engineering 10
4.3 Model Development 11
4.4 System Architecture and Deployment 12

Chapter 5: Results and Discussion 13

5.1 Evaluation Metrics 13
5.2 Performance Metrics 13
5.3 Comparative Analysis 14
5.4 Model Selection Reasoning 15
5.5 Deployment Screenshots 15

Chapter 6: Conclusion and Future Scope 18

6.1 Conclusion 18
6.2 Limitations 18
6.3 Future Scope 18

References 20

Chapter 1: Introduction

1.1 Background

Employee attrition, the gradual loss of employees over time through resignation, retirement, or termination, is one of the most persistent challenges faced by human resource departments. Replacing an employee is expensive: it involves recruitment costs, onboarding time, lost productivity, and the loss of institutional knowledge. Studies in HR analytics consistently show that voluntary attrition can cost an organization anywhere from half to twice an employee's annual salary, depending on the role's seniority and specialization.

With the growth of workforce analytics, organizations increasingly rely on historical employee data to identify at-risk employees before they resign, rather than reacting after the fact. This project applies supervised machine learning to the IBM HR Analytics Employee Attrition dataset to build a predictive system that flags employees with a high probability of leaving, so that HR teams can intervene early through targeted retention strategies.

1.2 Problem Statement

Given historical employee records covering demographics, compensation, job role, satisfaction scores, and tenure, the goal is to build a binary classification model that predicts whether a given employee is likely to leave the organization (Attrition = Yes) or stay (Attrition = No), and to deploy this model as an interactive web application that HR personnel can use without writing any code.

1.3 Objectives

- Perform thorough data preprocessing and exploratory data analysis (EDA) on the IBM HR Attrition dataset.
- Engineer meaningful new features that capture tenure, compensation-experience ratio, and satisfaction patterns.
- Train and compare four classification models: Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- Handle class imbalance using SMOTE (Synthetic Minority Oversampling Technique), and directly test it against simple class-weighting.
- Evaluate models using Accuracy, Precision, Recall, F1-Score, and ROC-AUC, and select the best model using a defensible tie-break rule rather than a single metric alone.
- Deploy the final model as a multi-page Streamlit web application supporting single-employee prediction, bulk CSV scoring, and interactive model insights.
- Optionally, wrap the same model in a standalone FastAPI + custom HTML/CSS/JS web application as an alternate, fully custom deployment.

1.4 Business Impact of Attrition

Beyond the direct recruitment cost, unplanned attrition disrupts ongoing projects, burdens remaining team members with extra workload, and can trigger a further wave of departures if morale drops. Roles that require domain-specific training, such as Research Scientists or Sales Executives in this dataset, are particularly expensive to replace, since a new hire typically needs several months to reach full productivity. A predictive system that surfaces high-risk employees weeks or months in advance gives HR and line managers a practical window to act, whether through a compensation review, a role change, or a direct conversation about career growth.

1.5 Scope of this Project

The scope of this capstone is limited to a single static snapshot of HR data (no time-series component) and to classical, well-understood machine learning models that are fast to train and reasonably interpretable, since explainability matters to HR stakeholders. The deployed system is intended as a decision-support tool for HR analysts, not an automated decision-maker; final retention decisions should always involve human judgement alongside the model's output.

> **Note**
> This system is a decision-support aid, not a replacement for HR judgement. Every prediction is a probability, not a verdict, and should be used to prioritize conversations, not to make unilateral decisions about a real employee.

Chapter 2: Literature Review

2.1 Existing Approaches

Employee attrition prediction has been studied extensively as a classic HR analytics classification problem. Most published approaches fall into three broad categories:

- Statistical / regression-based approaches. Logistic Regression remains a strong, interpretable baseline for HR attrition tasks, since stakeholders often need to understand which factors drive a prediction, not just the prediction itself.
- Tree-based ensemble approaches. Random Forest and Gradient Boosting methods (including XGBoost and LightGBM) are widely reported to outperform linear models on this dataset in raw accuracy, since they capture non-linear interactions between features such as overtime, income, and satisfaction.
- Deep learning approaches. On small tabular datasets such as this one (1,470 rows), neural networks tend to overfit and rarely outperform well-tuned tree ensembles, which is why this project focuses on classical ML models instead.

2.2 Research Studies

Publicly available analyses of the IBM HR Analytics dataset (originally released by IBM as a fictional-but-realistic dataset for benchmarking HR analytics techniques) consistently identify OverTime, MonthlyIncome, Age, TotalWorkingYears, and JobSatisfaction as the strongest predictors of attrition. A recurring finding across these studies is that the dataset is imbalanced, with only around 16% of employees marked as leavers, which pushes most practitioners toward oversampling techniques like SMOTE or class-weighting instead of accuracy alone as the evaluation criterion.

This project builds on those established findings but adds its own feature engineering (TenureRatio, IncomePerYearExp, PromotionGap, and AvgSatisfaction) to test whether combining raw HR fields into ratio-based indicators improves model discrimination.

2.3 Research Gap Addressed

Many public notebooks on this dataset optimize purely for accuracy, which is misleading given the roughly 5:1 class imbalance: a trivial always-predict-'No' baseline already scores around 84% accuracy while catching zero real leavers. This project explicitly optimizes model selection around F1-score and Recall instead, and documents the trade-off in Chapter 5, which is a gap in how several public analyses of this dataset are presented.

Chapter 3: Methodology

3.1 Dataset Description

The dataset used is the IBM HR Analytics Employee Attrition dataset, containing 1,470 employee records across 35 attributes with no missing values and no duplicate rows. The target variable, Attrition, has two classes: 1,233 'No' and 237 'Yes', giving an imbalance ratio of roughly 5.2 : 1.

Attribute	Description
Age	Employee age in years
Attrition	Target variable: whether the employee left (Yes/No)
Business Travel	Frequency of business travel
Daily Rate / Monthly Rate / Monthly Income / Hourly Rate	Compensation-related figures
Department / Job Role / Job Level	Organizational placement
DistanceFromHome	Commute distance in kilometres
Education / Education Field	Qualification level and field of study
Environment Satisfaction / Job Satisfaction / Relationship Satisfaction / Work Life Balance	Self-reported satisfaction scores (1-4)
Overtime	Whether the employee regularly works overtime
Total Working Years / Years At Company / Years In Current Role / Years Since Last Promotion / Years With Current Manager	Tenure-related fields
Stock Option Level / Percent Salary Hike / Performance Rating	Incentive and performance indicators

3.2 Data Preprocessing

- Removed four constant / non-informative columns: Employee Count, Standard Hours, Over18, and Employee Number (identifier, not predictive).
- Verified there were zero missing values and zero duplicate rows, so no imputation or deduplication was required.
- Encoded the target variable Attrition as 1 (Yes) / 0 (No).
- Label-encoded seven categorical columns: Business Travel, Department, Education Field, Gender, Job Role, Marital Status, and Overtime.
- Scaled all features using StandardAero before feeding them to the models, since Logistic Regression is sensitive to feature scale.

3.3 Algorithms Used

3.3.1 Logistic Regression

A linear model that estimates the probability of attrition as a sigmoid function of a weighted sum of input features. Chosen as an interpretable baseline; trained with mixite=1000 to ensure convergence.

3.3.2 Decision Tree

A single tree that recursively splits the data on the feature/threshold that most reduces impurity at each node. Included as a fast, fully interpretable baseline and to test whether a simple tree could match the ensemble models; depth was capped at 6 to limit overfitting a dataset of this size.

3.3.3 Random Forest

An ensemble of decision trees trained on bootstrapped samples with random feature subsets, then aggregated by majority vote. Hyperparameters (estimators, max-depth, min-samples split) were tuned using 3-fold Grid Search-CV optimizing for F1-score, with class weight='balanced' to further counter class imbalance.

3.3.4 XG-Boost

A gradient boosting framework that builds trees sequentially, each correcting the errors of the previous ensemble. Configured with 300 estimators, max-depth=5, and a learning rate of 0.05 to balance bias and variance on this relatively small dataset.

3.4 Handling Class Imbalance

Since only 16.1% of employees in the dataset actually left training directly on the raw data biases every model toward predicting the majority 'No' class. SMOTE (Synthetic Minority Oversampling Technique) addresses this by generating new synthetic minority-class samples through interpolation between existing minority-class neighbours, rather than simply duplicating rows. SMOTE was applied only after the train/test split, and only to the training set, so that the test set remains a realistic, untouched sample of the true population; applying SMOTE before splitting would leak synthetic near duplicates into the test set and inflate reported performance.

To verify SMOTE was actually worth the added complexity, a fifth model was trained as a direct comparison: the same Logistic Regression algorithm, on the same encoded/engineered features, but using scikit-lean's class weight='balanced' option on the original unbalanced data instead of SMOTE oversampling. The result, discussed in Chapter 5, showed SMOTE gave a meaningfully better recall/F1 trade-off, confirming the added preprocessing step earns its place in the pipeline.

> **Key Insight**
>
> SMOTE oversampling beat simple class-weighting on this dataset: 76.6% recall vs 72.3%, at a similar ROC-AUC. The extra preprocessing step is worth it.

Chapter 4: Implementation

4.1 Exploratory Data Analysis

Univariate analysis of the target variable confirmed the expected class imbalance:

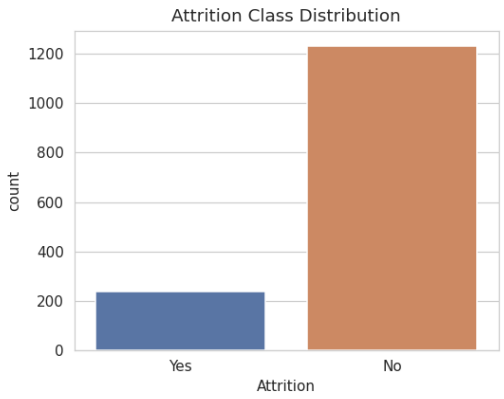


Figure 4.1: Attrition class distribution (1,233 No vs 237 Yes)

Bivariate analysis of Age against Attrition showed that younger employees, particularly those under 30, leave at a noticeably higher rate than older, more established employees:

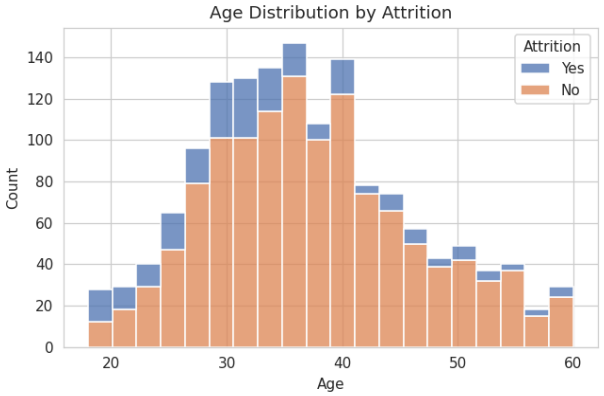


Figure 4.2: Age distribution split by attrition

Employees who left tend to have a lower median Monthly Income than those who stayed, and OverTime shows one of the clearest bivariate splits in the entire dataset: employees who work overtime leave at a substantially higher rate.

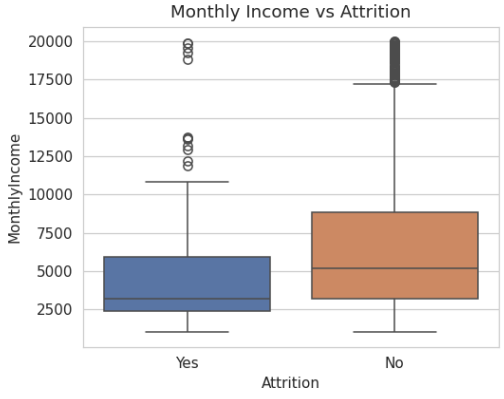


Figure 4.3: Monthly Income vs Attrition

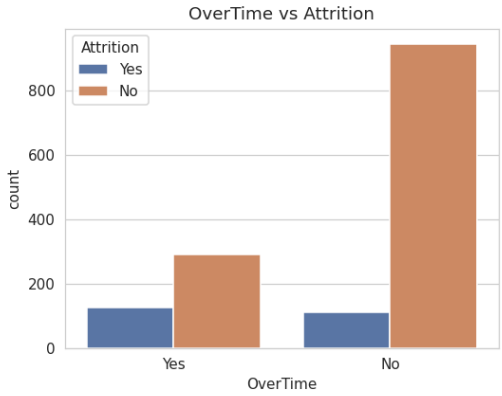


Figure 4.4: Overtime vs Attrition

The correlation heatmap over all numeric features highlights expected clusters: tenure-related fields (Years At Company, Years In CurrentRole, Years With Current Manager) are strongly correlated with each other, and Job Level correlates strongly with Monthly Income and Total Working Years.

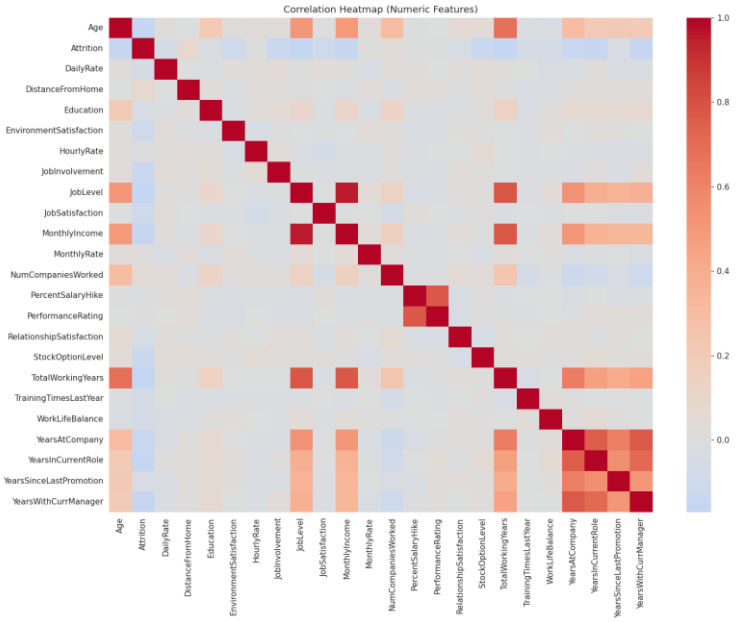


Figure 4.5: Correlation heatmap of numeric features

Grouping attrition rate by Job Role reveals that Sales Representatives and Laboratory Technicians have disproportionately higher attrition rates compared to Managers and Research Directors, who show the lowest attrition:

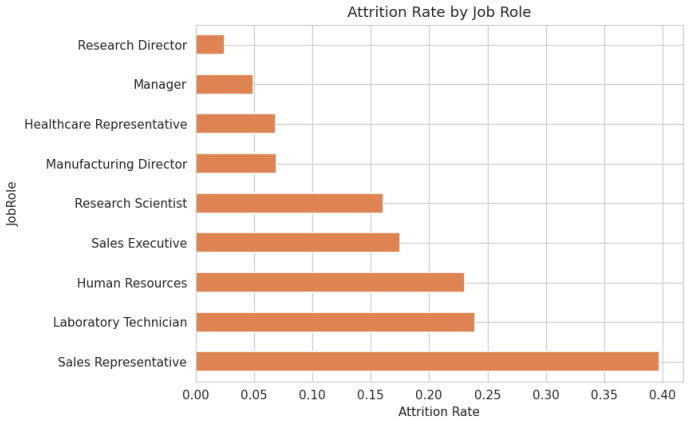


Figure 4.6: Attrition rate by Job Role

Work-Life Balance also shows a visible relationship with attrition, with employees reporting the lowest balance score (1) leaving proportionally more often:

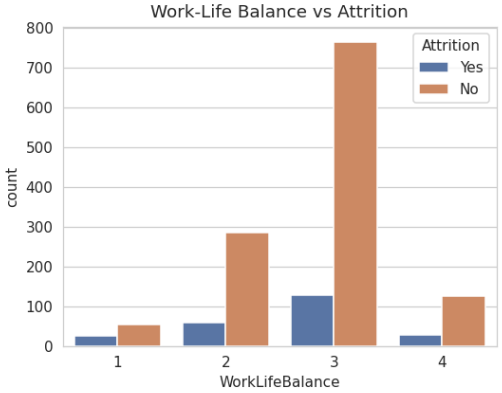


Figure 4.7: Work-Life Balance vs Attrition

4.2 Feature Engineering

Four new features were engineered to capture patterns that raw columns do not directly express:

- Tenure Ratio = Years At Company / (Total Working Years + 1). Captures how much of an employee's total career has been spent at this company.
- Income Per Year Exp = Monthly Income / (Total Working Years + 1). A rough proxy for whether an employee is being compensated fairly relative to their experience.
- Promotion Gap = Years At Company − Years Since Last Promotion. Highlights employees who may feel stagnant in their career progression.
- Av Satisfaction: the mean of Job Satisfaction, Environment Satisfaction, Relationship Satisfaction, and Work Life Balance, combined into a single composite indicator.

A quick Random Forest feature-importance check confirmed that several engineered and raw features rank among the most predictive:

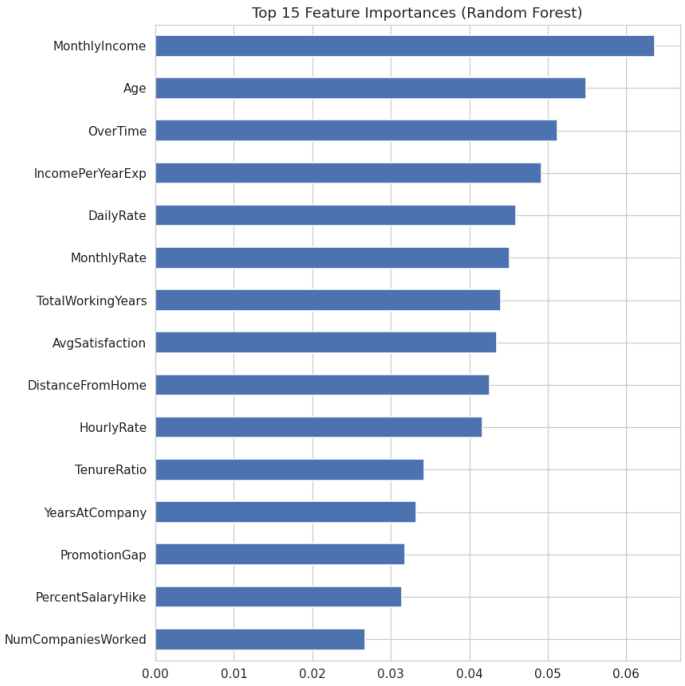


Figure 4.8: Top 15 feature importances

Rank	Feature	Importance
1	Monthly Income	0.0636
2	Age	0.0549
3	Overtime	0.0512
4	Income Per Year Exp (engineered)	0.0492
5	Daily Rate	0.0460
6	Monthly Rate	0.0452
7	Total Working Years	0.0440
8	Av Satisfaction (engineered)	0.0435
9	DistanceFromHome	0.0426
10	Hourly Rate	0.0416

Two of the four engineered features, Income Per Year Exp and Av Satisfaction, rank inside the top 10 most important features, which supports the decision to include them alongside the raw dataset columns.

4.3 Model Development

The dataset was split 80/20 into training and test sets using stratified sampling to preserve the original class ratio in both splits. SMOTE was then applied only to the training data, oversampling the minority ('Yes') class to match the majority class (986 vs 986 after balancing), while the test set was left untouched to give an honest, realistic evaluation. Logistic Regression, Decision Tree, Random Forest, and XG-Boost were all trained on this SMOTE-balanced training set and evaluated on the same untouched test set. A fifth variant (Logistic Regression with class weight='balanced', no SMOTE) was trained on the original unbalanced training set as a direct control, discussed in Chapter 5.

4.4 System Architecture and Deployment

The end-to-end system follows a standard ML deployment pattern, expanded into two independent deployment options that share a single source of truth for all prediction logic:

- Data Layer: Dataset/HR-Employee-Attrition.csv
- Processing & Training Layer: Notebook/Employee_Attrition_Prediction.ipynb (Google Colab notebook, executed end-to-end with saved outputs)
- Model Artifact Layer: Model/ (best_model.pkl, scaler.pkl, label_encoders.pkl, feature_names.json, model_comparison.csv)
- Shared Logic Layer: Streamlit_App/model_utils.py, containing the feature engineering, encoding, and single/batch prediction functions used by both deployments below, so results can never drift out of sync between them.

Deployment Option A, Multi-Page Streamlit Application (primary deliverable): organized as five pages matching how an HR analyst would actually use the tool. Predict is the main form, grouped into four tabs (Personal & Role, Compensation, Satisfaction & Work-Life, Tenure & History) instead of one long scroll, returning a result card with a probability gauge and short rule-based context notes. Batch Prediction lets a user upload a CSV of many employees and score all of them at once, with a downloadable template, clear error messages, and a downloadable results file. Compare places two full copies of the form side-by-side so two employees can be scored and directly contrasted in one view. Model Insights presents an interactive view of the model comparison table/chart and key EDA figures, so the reasoning behind model selection is visible inside the app itself. About holds project and author information. The interface uses a custom navy-and-red theme matching this report and the presentation deck; colors, text, and the entire form are defined in two separate configuration files (theme.py and fields_config.py) so the interface can be restyled or extended without touching the prediction code.

Deployment Option B, Advanced Web App (optional, non-required): the same model is also exposed through a FastAPI backend (endpoints for single prediction, batch prediction, model metadata, and a health check) paired with a hand-built HTML/CSS/JS frontend styled independently of Streamlit's component library, including interactive Chart.js visualizations. This demonstrates the model can be served through a fully custom interface without being tied to any particular Python UI framework. Both deployments call the exact same underlying functions, so a prediction for identical inputs is guaranteed to match between them.

Chapter 5: Results and Discussion

5.1 Evaluation Metrics

Given the class imbalance, five complementary metrics were used to judge each model, defined in terms of True Positives (TP), True Negatives (TN), False Positives (FP), and False Negatives (FN):

Metric	Formula	What it Measures
Accuracy	(TP + TN) / Total	Overall fraction of correct predictions
Precision	TP / (TP + FP)	Of predicted leavers, how many actually left
Recall	TP / (TP + FN)	Of actual leavers, how many were correctly caught
F1-Score	2 x (Precision x Recall) / (Precision + Recall)	Harmonic balance of precision and recall
ROC-AUC	Area under the ROC curve	Model's ability to rank leavers above stayers across thresholds

5.2 Performance Metrics

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Logistic Regression	0.762	0.379	0.766	0.507	0.807
Decision Tree	0.810	0.435	0.638	0.517	0.736
Random Forest	0.847	0.538	0.298	0.384	0.794
XGBoost	0.864	0.640	0.340	0.444	0.784
Logistic Regression (class_weight, no SMOTE)	0.752	0.362	0.723	0.482	0.813

Table 5.1: Test-set performance of all five trained model variants

Accuracy alone is misleading on an imbalanced dataset like this: a model that always predicts 'No' would already score roughly 84% accuracy while catching zero real leavers. This is why F1-score and Recall are given more weight in model selection. Recall measures how many actual leavers the model successfully identifies, which is the metric that matters most from an HR intervention standpoint.

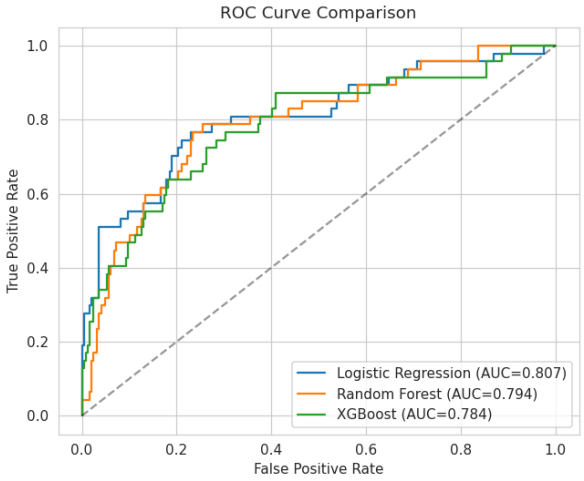


Figure 5.1: ROC curve comparison (Logistic Regression, Random Forest, XGBoost, from the original 3-model training run; see Table 5.1 for the full 5-model comparison including Decision Tree and the SMOTE-ablation control)

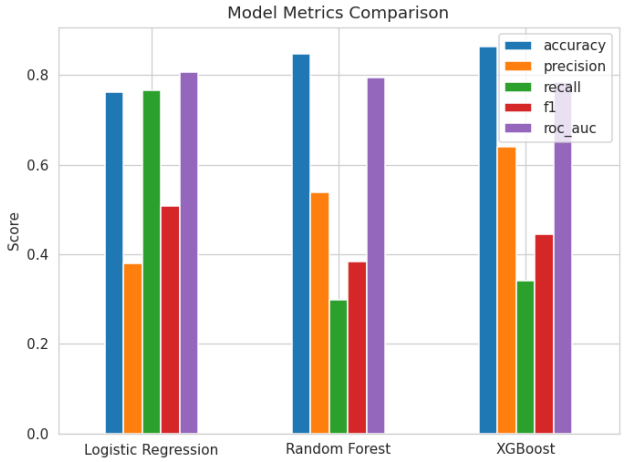


Figure 5.2: Side-by-side metric comparison

5.3 Comparative Analysis

Logistic Regression achieved the lowest raw accuracy (76.2%) but by far the highest recall (76.6%) among the SMOTE-trained models, along with a strong F1-score (0.507). In practice, this means it correctly flags roughly three out of every four employees who actually leave, at the cost of a higher false-positive rate. Decision Tree scored the single highest F1 (0.517) with a moderate recall (63.8%), but its ROC-AUC (0.736) trails well behind Logistic Regression's (0.807). This means it ranks leavers versus stayers noticeably less reliably across different decision thresholds, and single trees also tend to be more prone to variance on a dataset this size. Random Forest and XGBoost achieved the highest accuracy (84.7% and 86.4%) and the best precision, but their recall collapsed to 29.8% and 34.0% respectively. Both tree ensembles miss the majority of real leavers despite class-weighting and SMOTE balancing, making them a poor fit for an HR early-warning use case.

The direct SMOTE-vs-class weight control confirms the preprocessing choice made in Section 3.4. Logistic Regression with class weight='balanced' alone (no SMOTE) reached a similar ROC-AUC (0.813) but a lower F1 (0.482) and lower recall (72.3%) than the SMOTE-trained version (0.507 F1, 76.6% recall). SMOTE's synthetic oversampling therefore gives a modest but real improvement over simply reweighting the existing imbalanced data.

> **Key Insight**
>
> Raw accuracy is the wrong metric to optimize here. XGBoost has the highest accuracy (86.4%) but catches barely a third of real leavers (34.0% recall) — Logistic Regression trades some accuracy for catching more than double that share.

5.4 Model Selection Reasoning

With five model variants to choose from, picking whichever has the single highest F1-score is not robust: a model that wins F1 by a hair while being far worse on ROC-AUC (a threshold-independent measure of how well a model ranks leavers above stayers across all cutoffs, not just the default 0.5) is a weaker overall choice. The rule applied instead: find the highest F1-score achieved by any model, treat every model within 0.02 of that score as “practically tied,” and among those tied models, deploy whichever has the best ROC-AUC.

Model	F1-Score	Within 0.02 of top F1?	ROC-AUC
Logistic Regression	0.507	Yes (top score)	0.807 ← highest among tied models
Decision Tree	0.517	Yes (highest F1 overall)	0.736

Decision Tree's F1-score (0.517) is technically the highest of all five models, and Logistic Regression's (0.507) falls within 0.02 of it, so the two are treated as tied on F1. Between them, Logistic Regression's ROC-AUC (0.807) decisively beats Decision Tree's (0.736), so Logistic Regression is the model that gets deployed, not because it "won" on every metric, but because it is the more robust choice among the models that were statistically tied on the metric that matters most for this use case. This selection logic is implemented directly in the training notebook rather than left as a manual judgement call, so the outcome is reproducible.

5.5 Deployment Screenshots

The selected model was serialized with jilbab and is served through the two deployments described in Section 4.4. The screenshots below show the current multi-page Streamlit application running locally.

The Predict page, with the hero header (deployed model, F1, ROC-AUC), the tabbed input form, and the sidebar showing model metrics:

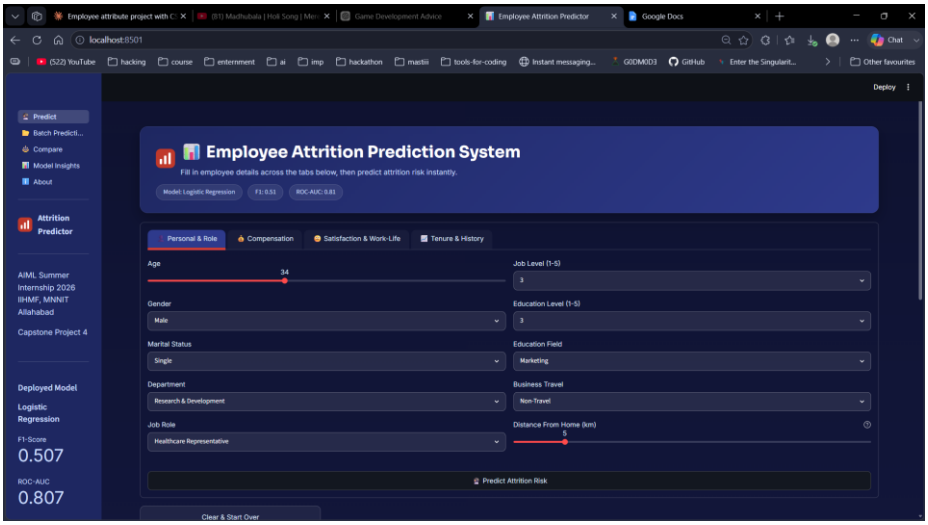


Figure 5.3: Predict page — hero header and tabbed input form

After clicking "Predict Attrition Risk," the result card, gauge visualization, contributing-context notes, and session history table all appear below the form:

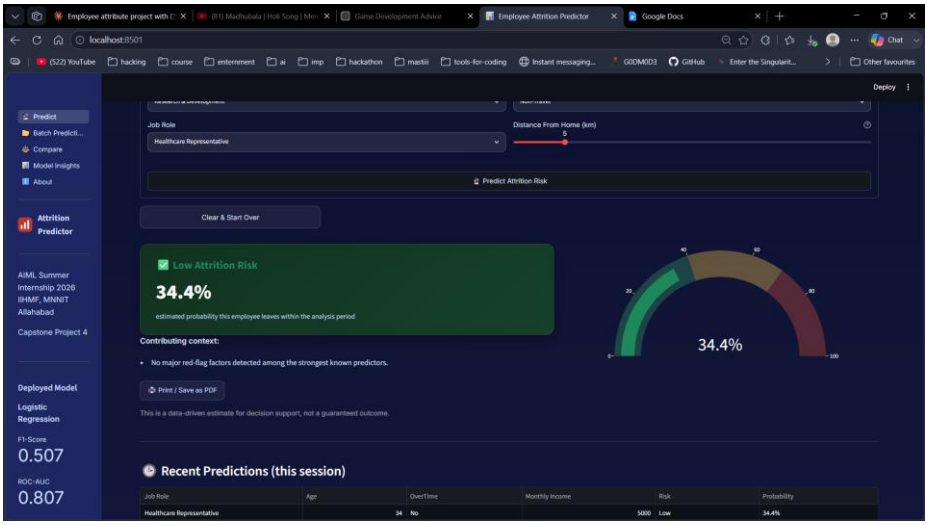


Figure 5.4: Predict page — live prediction result (Low Attrition Risk, 34.4% probability) with gauge and session history

The Batch Prediction page after uploading a CSV and scoring it, showing the summary metrics and per-row results:

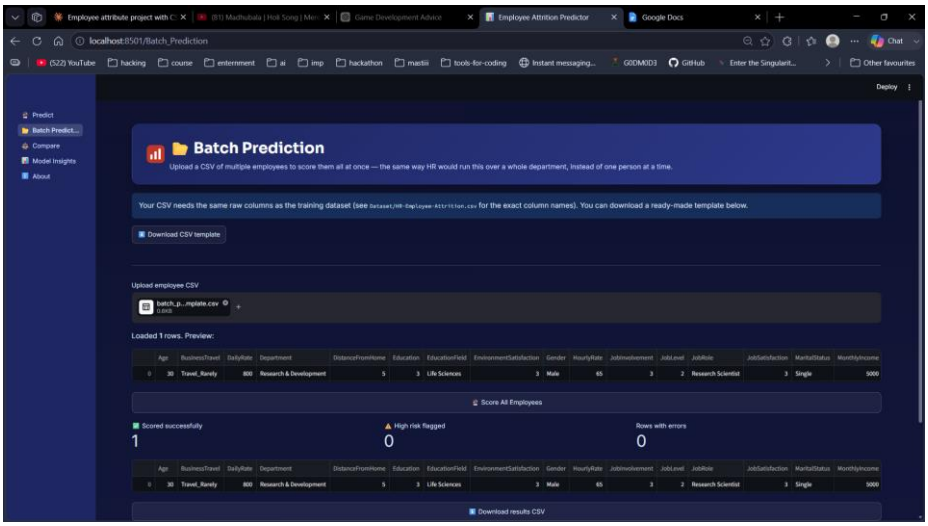


Figure 5.5: Batch Prediction page — scored results with summary metrics and results table

The Model Insights page, showing the interactive 5-model comparison table (with per-column best scores highlighted) and the accompanying bar chart:

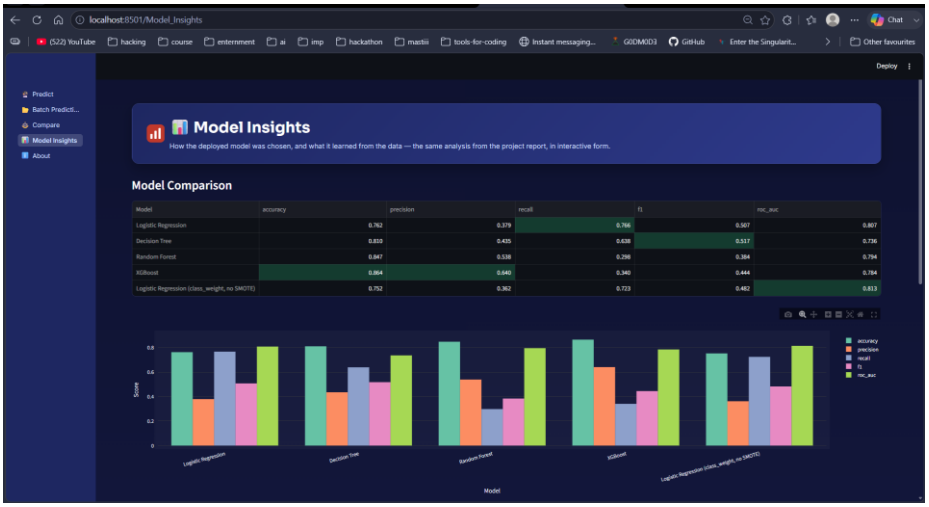


Figure 5.6: Model Insights page — model comparison table and chart

Chapter 6: Conclusion and Future Scope

6.1 Conclusion

This project built a complete, end-to-end employee attrition prediction pipeline: data preprocessing, exploratory data analysis, feature engineering, class-imbalance handling with SMOTE (validated directly against class-weighting), training and comparing four classification models plus a SMOTE-ablation control, and deploying the best-performing model through two independent interfaces, a multi-page Streamlit application (Predict, Batch Prediction, Compare, Model Insights, About) and an optional custom FastAPI + HTML/CSS/JS web application. The analysis confirmed several well-known HR risk factors, overtime, low relative compensation, frequent travel, and low job/environment satisfaction, as the strongest drivers of attrition in this dataset, while also demonstrating that model selection for HR use cases should prioritize recall and ROC-AUC over raw accuracy or a single metric in isolation.

> **Project Takeaway**
> A defensible model choice beats a blindly "optimal" one. Logistic Regression isn't the top scorer on any single metric here — it's the deployed model because its trade-offs are the ones that make sense for an HR early-warning tool.

6.2 Limitations

> **Caution**
> This model has modest precision (37.9%) — most "high risk" flags will NOT actually leave. Treat predictions as a prioritization signal for retention conversations, never as a standalone reason for any employment decision.

- The dataset is a single static snapshot (1,470 employees from one point in time), so the model reflects historical patterns and may not generalize to organizations with very different cultures or compensation structures.
- Precision remains modest for the deployed Logistic Regression model (37.9%), meaning a meaningful share of flagged employees will not actually leave; the tool is best used to prioritize retention conversations, not as an automatic decision-maker.
- The dataset does not include external factors such as macroeconomic conditions, competitor offers, or personal circumstances, all of which influence real-world attrition.

6.3 Future Scope

- Add model explainability (e.g. SHAP values) so HR staff can see exactly why a given employee was flagged as high-risk, not just the final probability.
- Experiment with stacking/ensembling Logistic Regression, Random Forest, and XGBoost to combine high recall with better precision.
- Collect longitudinal, multi-year data to enable time-to-attrition survival modeling instead of a single static snapshot.
- Integrate the model into an HR information system (HRIS) with scheduled batch scoring and automated alerts to managers.

- Explore fairness auditing to ensure the model does not disproportionately flag employees based on gender, age, or marital status.

References

1. IBM HR Analytics Employee Attrition & Performance Dataset. IBM Watson Analytics / Kaggle.

2. Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research.

3. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.

4. Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

5. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.

6. Streamlit Documentation. =>https://docs.streamlit.io/

7. FastAPI Documentation. =>https://fastapi.tiangolo.com/

8. Chart.js Documentation.=> https://www.chartjs.org/

9. github-link =>https://github.com/architcoder1234/EmployeeAttritionPrediction